PCE Agent Feature Flag — Complete Implementation Guide

> **Goal:** Add a single `AGENTS_ENABLED` environment variable that completely enables or disables all agentic AI features. When `false`, agents consume zero resources, produce zero LLM costs, and their Docker service does not start.

Files to Modify (Backend)

1. `app/config.py` — Add the master switch

Add this field inside the `Settings` class, after the `SCRAPER_TIMEOUT_MS` field:

```
    # --- Agentic AI Orchestrator ---
    # Master switch: if False, all agent tasks become no-ops and the
    # agent_orchestrator container is not started.
    AGENTS_ENABLED: bool = Field(default=False)
```

Full context snippet:

```
    SCRAPER_TIMEOUT_MS: int = Field(default=15000, ge=5000, le=60000)

    # --- Agentic AI Orchestrator ---
    AGENTS_ENABLED: bool = Field(default=False)

    @property
    def DATABASE_URL(self) -> str:
```

---

2. `app/celery_app.py` — Conditionally register agent beat schedules

Add import at the top:

```
from app.config import settings
```

Replace the entire `create_celery_app()` function with this version (it keeps every existing schedule and only adds agent tasks when enabled):

```
def create_celery_app() -> Celery:
    app = Celery(
        "price_comparison_engine",
        broker=os.environ.get("REDIS_URL", "redis://localhost:6379/0"),
        backend=os.environ.get("REDIS_URL", "redis://localhost:6379/0"),
        include=[
            "app.tasks",
            "app.workers.scraper_worker",
            "app.workers.email_worker",
            "app.tasks_agents",          # NEW: agent task module
        ],
    )

    app.conf.update(
```

```
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="UTC",
    enable_utc=True,
    task_track_started=True,
    task_time_limit=300,
    task_soft_time_limit=240,
    worker_prefetch_multiplier=1,
    worker_max_tasks_per_child=50,
)

# ---------------------------------------------------------------------
# Beat Schedule (Periodic Tasks)
# ---------------------------------------------------------------------
app.conf.beat_schedule = {
    "scrape-active-vendors": {
        "task": "app.tasks.process_vendor_scrape",
        "schedule": 300.0,
        "options": {"queue": "default"},
    },
    "price-drop-emails": {
        "task": "app.tasks.trigger_price_drop_emails",
        "schedule": 600.0,
        "options": {"queue": "default"},
    },
    "price-drop-push-notifications": {
        "task": "app.tasks.check_price_drops_and_queue_push",
        "schedule": 300.0,
        "options": {"queue": "high_priority"},
    },
    "reset-sponsored-budgets": {
        "task": "app.tasks.reset_daily_sponsored_budgets",
        "schedule": 86400.0,
        "options": {"queue": "default"},
    },
    "archive-old-clicks": {
        "task": "app.tasks.archive_old_clicks",
        "schedule": 604800.0,
        "options": {"queue": "default"},
    },
}

# ---------------------------------------------------------------------
# Agent Beat Schedules (ONLY when AGENTS_ENABLED=true)
# ---------------------------------------------------------------------
if settings.AGENTS_ENABLED:
    app.conf.beat_schedule.update({
        "agent-entity-resolution": {
            "task": "app.tasks_agents.run_entity_resolution_pipeline",
            "schedule": 300.0,   # every 5 min
```

```
                "options": {"queue": "default"},
            },
            "agent-selector-health": {
                "task": "app.tasks_agents.run_selector_health_check",
                "schedule": 3600.0,  # every 1 hour
                "options": {"queue": "default"},
            },
        })

    # ------------------------------------------------------------------
    # Task Routes
    # ------------------------------------------------------------------
    app.conf.task_routes = {
        "app.tasks.send_fcm_push_notification": {"queue": "high_priority"},
        "app.tasks.process_vendor_scrape": {"queue": "default"},
        "app.tasks.trigger_price_drop_emails": {"queue": "default"},
        "app.tasks.check_price_drops_and_queue_push": {"queue": "high_priority"},
        "app.tasks.reset_daily_sponsored_budgets": {"queue": "default"},
        "app.tasks.archive_old_clicks": {"queue": "default"},
    }

    # Agent task routes (ONLY when enabled)
    if settings.AGENTS_ENABLED:
        app.conf.task_routes.update({
            "app.tasks_agents.*": {"queue": "default"},
        })

    app.conf.result_expires = 3600
    app.conf.result_extended = True

    return app
```

---

3. `app/tasks_agents.py` — NEW FILE (agent task module with guard)

Create this file at `app/tasks_agents.py`. Every agent task checks `AGENTS_ENABLED` at entry and returns immediately if disabled.

```
"""
Agentic AI task definitions.

All tasks here are gated by the AGENTS_ENABLED setting.
When False, tasks return immediately with a 'skipped' status
and consume zero LLM credits / zero compute.
"""
from celery.utils.log import get_task_logger

from app.celery_app import celery_app
from app.config import settings

logger = get_task_logger(__name__)
```

```python
def _agents_disabled_guard() -> bool:
    """Returns True if agents are disabled (task should abort)."""
    if not settings.AGENTS_ENABLED:
        logger.info("[AGENT] AGENTS_ENABLED=false — task skipped.")
        return True
    return False


# ============================================================
# Pipeline 1: Entity Resolution (match scraped offers to products)
# ============================================================

@celery_app.task(bind=True, max_retries=2, default_retry_delay=60)
def run_entity_resolution_pipeline(self):
    """
    Fetches pending_review offers, runs hybrid vector + text search,
    calls LLM for match/new-product/escalate decision, and writes back.
    """
    if _agents_disabled_guard():
        return {"status": "skipped", "reason": "agents_disabled"}

    logger.info("[AGENT] Starting entity resolution pipeline...")
    # TODO: Wire in real logic from Data Quality Agent when ready.
    # from app.agents.entity_resolution import EntityResolutionPipeline
    # pipeline = EntityResolutionPipeline()
    # return pipeline.run()
    return {"status": "success", "processed": 0, "note": "placeholder"}


# ============================================================
# Pipeline 2: Selector Health (test vendor CSS selectors via Playwright)
# ============================================================

@celery_app.task(bind=True, max_retries=2, default_retry_delay=120)
def run_selector_health_check(self):
    """
    Playwright-tests every active vendor CSS selectors.
    If a selector is broken, uses LLM to suggest a fix.
    Auto-applies if confidence > 0.6.
    """
    if _agents_disabled_guard():
        return {"status": "skipped", "reason": "agents_disabled"}

    logger.info("[AGENT] Starting selector health check...")
    # TODO: Wire in real logic from Data Quality Agent when ready.
    # from app.agents.selector_health import SelectorHealthChecker
    # checker = SelectorHealthChecker()
    # return checker.run()
    return {"status": "success", "checked": 0, "fixed": 0, "note": "placeholder"}
```

```python
# ============================================================
# Pipeline 3: Data Quality Audit (future expansion)
# ============================================================

@celery_app.task
def run_data_quality_audit():
    """Placeholder for future data-quality audits (outliers, stale prices)."""
    if _agents_disabled_guard():
        return {"status": "skipped", "reason": "agents_disabled"}
    logger.info("[AGENT] Running data quality audit...")
    return {"status": "success", "issues_found": 0}
```

---

4. `docker-compose.yml` — Uncomment agent service with `profiles`

Replace the commented-out `agent_orchestrator` block with this. The `profiles: ["agents"]` line means the service **only starts when you explicitly ask for it**.

```yaml
  # ========================================
  # 10. AGENT ORCHESTRATOR (Agentic AI)
  #      Start with: docker compose --profile agents up -d
  # ========================================
  agent_orchestrator:
    profiles: ["agents"]
    build:
      context: ./agents
      dockerfile: Dockerfile
    container_name: pce_agent_orchestrator
    restart: unless-stopped
    env_file: .env
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - AGENTS_ENABLED=${AGENTS_ENABLED:-false}
    volumes:
      - ./agents/prompts:/app/prompts
      - ./agents/tools:/app/tools
    depends_on:
      - api
      - db
      - redis
    networks:
      - app_network
```

Key point: Without `--profile agents`, `docker compose up` completely ignores this service. No container is built, no memory is used, no LLM keys are touched.

---

5. `app/main.py` — Add agent health endpoint

Add this endpoint inside `app/main.py`, right after the existing `/health` endpoint:

```python
@app.get("/health/agents")
async def agent_health_check():
    """Returns agent orchestrator status for dashboards & admin UI."""
    return {
        "agents_enabled": settings.AGENTS_ENABLED,
        "status": "active" if settings.AGENTS_ENABLED else "standby",
        "message": (
            "Agent orchestrator is ready. Set AGENTS_ENABLED=true to activate."
            if not settings.AGENTS_ENABLED
            else "Agent tasks are running."
        ),
    }
```

---

6. `.env.example` — Add the flag

Add this line anywhere in `.env.example`:

```
# --- Agentic AI ---
# Set to true only when you are ready to run LLM-powered agents.
# When false, agent containers do not start and agent tasks are no-ops.
AGENTS_ENABLED=false
```

Files to Modify (Frontend — Flutter)

7. `lib/services/api_service.dart` — Point to your backend

Change the hardcoded `baseUrl` from localhost to your actual backend URL.

For local development (backend + emulator on same machine):

```dart
  static const String baseUrl = 'http://10.0.2.2:8000/api/v1';  // Android emulator
  // static const String baseUrl = 'http://localhost:8000/api/v1'; // iOS simulator
```

For production / Cloudflare Tunnel:

```dart
  static const String baseUrl = 'https://api.yourdomain.com/api/v1';
```

> **Android emulator note:** `10.0.2.2` is the special IP that routes to the host machine localhost. If you use a real device, use your machine LAN IP (e.g., `192.168.1.42:8000`).

8. `lib/services/pce_api_service.dart` — Same URL update

```dart
  static const String _baseUrl = 'http://10.0.2.2:8000/api/v1';  // Android emulator
  // static const String _baseUrl = 'https://api.yourdomain.com/api/v1'; // Production
```

How to Enable Agents Later (One-Command Flip)

When your app has user traction and you want to turn on agents:

1. **Set the flag:**

```
# In your .env file
AGENTS_ENABLED=true
```

2. **Restart core services to pick up the new env:**

```
docker compose up -d
```

3. **Start the agent container (uses the `agents` profile):**

```
docker compose --profile agents up -d
```

4. **Verify:**

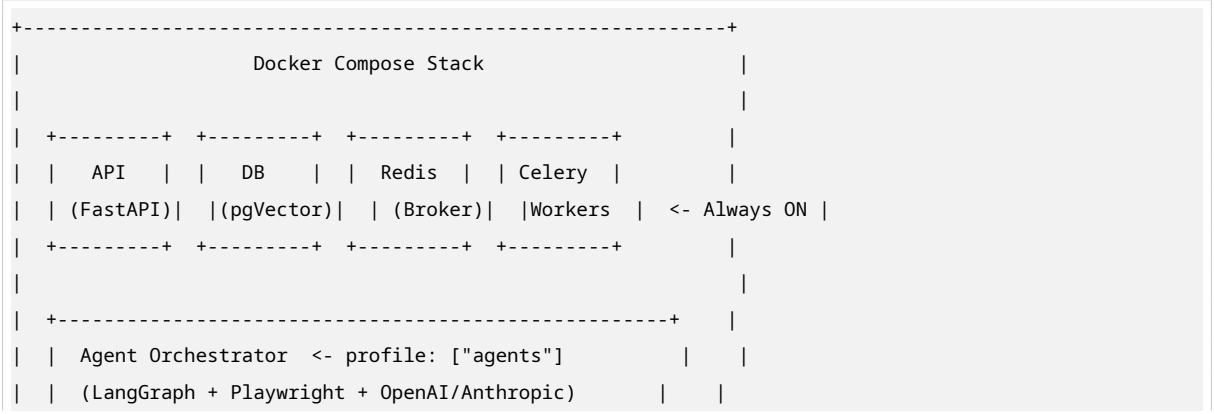
```
curl http://localhost:8000/health/agents
# Expected: {"agents_enabled": true, "status": "active", ...}
```

5. **Monitor agent tasks in Flower:** Open http://localhost:5555 and look for `app.tasks_agents.*` tasks in the queue.

Cost Impact Summary

State	AGENTS_ENABLED	Agent Container	LLM Calls	Beat Schedules	Monthly Cost
Disabled (default)	`false`	Not running	Zero	Not registered	Rs. 0
Enabled	`true`	Running	Per schedule	Active	~Rs. 21,600/mo (GPT-4o-mini @ 50 offers x 288 runs/day)

Architecture Diagram



```
|  |  Only starts when: docker compose --profile agents   |    |
|  +-------------------------------------------------------+    |
|                          |                                    |
|              Gated by AGENTS_ENABLED=true                     |
+---------------------------------------------------------------+
```